

| 2026-04-01

| Сверточные нейронные сети, регуляризация и перенос обучения

Многослойные перцептроны плохо масштабируются на реальные задачи работы с изображениями: изображение в RGB даст нам $224 \times 224 \times 3 = 150528$ входных признаков. В одном полносвязном скрытом слое из 1 тыс. нейронов будет **150 528 000 параметров** — и только на первом слое.

Как следствие:

1. Катастрофическое переобучение при типичных размерах датасетов.
2. MLP одинаково «смотрит» на соседние пиксели и на пиксели в противоположных углах, пространственная структура не учитывается.
3. Трансляционная инвариантность отсутствует: котик в левом углу и котик в правом — для MLP это разные объекты.

| Компьютерное зрение

Все вышеуказанные проблемы критичны для задач компьютерного зрения.

Компьютерное зрение (CV) — процесс интерпретации визуального сигнала (изображение или видео) посредством компьютера (ЭВМ).

| Основные сложности при решении задач CV

1. Разные ракурсы одного и того же объекта
2. Разное освещение
3. Перекрывание объектов (окклюзия)
4. Разный масштаб
5. Искажения и артефакты
6. Зашумленность фона (много лишних деталей в кадре)
7. Высокая внутриклассовая вариативность (стул, диван — выглядят очень по-разному, но все при этом может быть классом «мебель»)

| Эволюция подходов

| 1. Ручные признаки

1. **Наивный Байес** — «есть объект / нет объекта» по пиксельным признакам.
2. **Bag of Features** — гистограмма локальных дескрипторов, заимствованная из NLP («мешок слов» / Bag of Words, переложенный на признаки изображения).

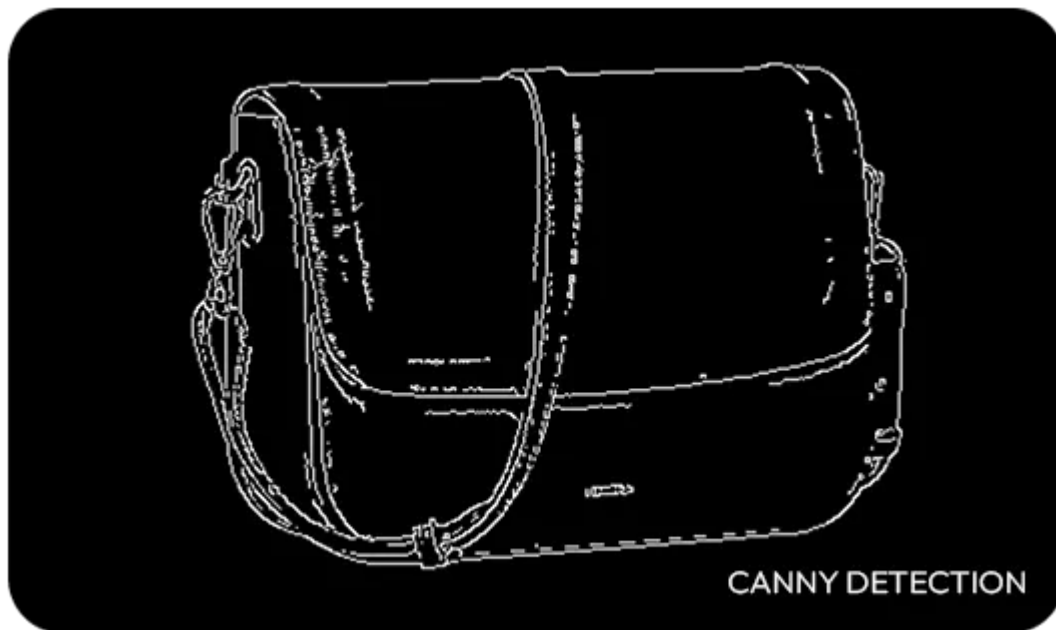
3. **SIFT (Scale-Invariant Feature Transform)** — ключевые точки, инвариантные к масштабу, повороту, освещению. Золотой стандарт до ~2012 года.



Ограничения SIFT: нет семантической интерпретации признаков; невозможно обеспечить локализацию и сегментацию; контринтуитивное разбиение объекта на компоненты.

2. Классическая обработка изображений

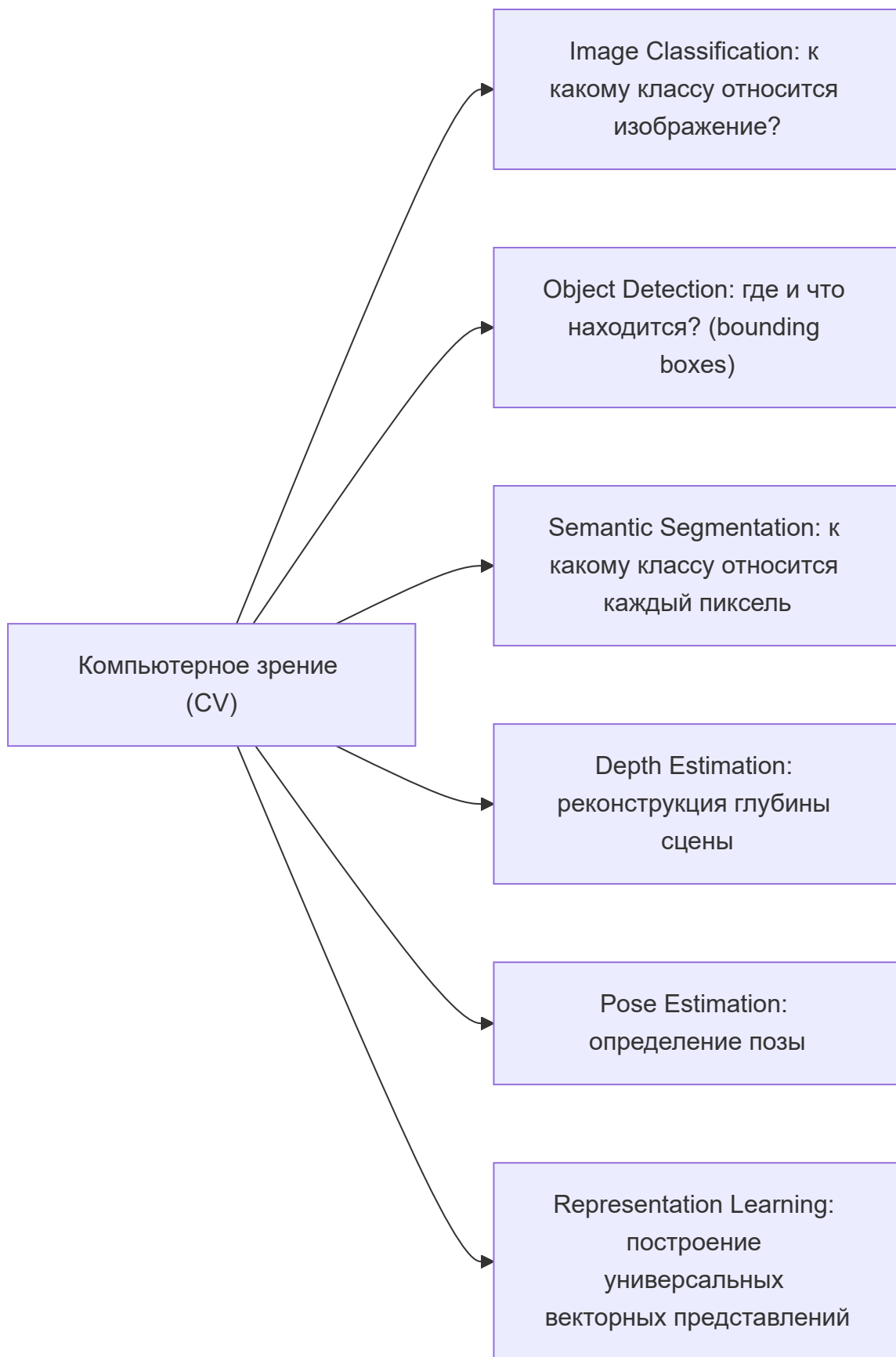
1. **Thresholding** — перевод изображения в бинарную матрицу по порогу яркости.
2. **Canny edge detector** — детекция границ объектов.



3. Глубокое обучение

С ~2012 года активно используется глубокое обучение, в первую очередь, сверточные нейронные сети (CNN), которые решили почти все ключевые проблемы предыдущих подходов, т.к. сеть сама учится извлекать признаки, инвариантные к трансформациям.

Прикладные задачи CV



1. **Image Classification** — к какому классу относится изображение?
2. **Object Detection** — где и что находится? (bounding boxes)
3. **Semantic Segmentation** — к какому классу относится каждый пиксель?
4. **Depth Estimation** — реконструкция глубины сцены
5. **Pose Estimation** — определение позы

| Свёрточные нейронные сети (Convolutional Neural Networks, CNN)

| Ключевые идеи

1. **Локальные связи** — каждый нейрон «видит» только небольшую область, т.н. рецептивное поле (receptive field).
2. **Разделяемые веса (weight sharing)** — одно и то же ядро применяется ко всем позициям.
 - Детектор горизонтального ребра работает одинаково в левом верхнем углу и в правом нижнем, что дает в результате **трансляционную инвариантность**.

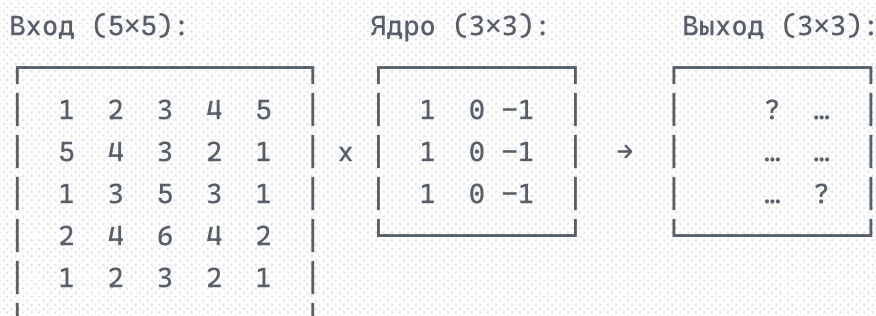
Tip

MLP: каждый нейрон подключен ко **ВСЕМ** пикселям

CNN: каждый нейрон подключен только к **ОКРЕСТНОСТИ** 3×3 или 5×5

| Операция свертки

Ядро скользит по изображению и на каждой позиции вычисляет скалярное произведение с увиденным локальным участком изображения.



Формула для одноканального изображения I и ядра K размером $k \times k$:

$$(I * K)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I[i + m, j + n] \cdot K[m, n]$$

Параметры свертки:

Параметр	Описание	Примеры значений
<code>kernel_size</code>	Размер ядра	3×3 (стандарт), 5×5
<code>stride</code>	Шаг скольжения	1 (без уменьшения), 2 (вдвое меньше выход)

Параметр	Описание	Примеры значений
<code>padding</code>	Добавление полей (нули по краям)	<code>padding=1</code> при <code>kernel=3</code> сохраняет размер
<code>in_channels</code>	Каналов на входе	3 для RGB (1 на цвет)
<code>out_channels</code>	Количество фильтров	32, 64, 128...

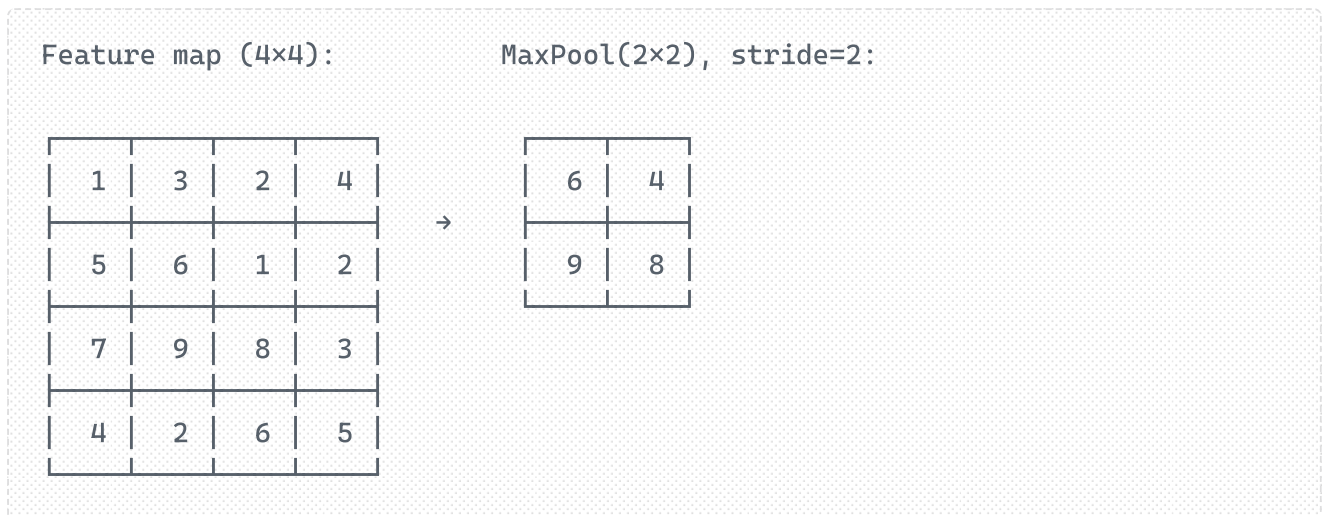
Для RGB (C_{in} каналов) ядро имеет размер $C_{in} \times k \times k$.

Один фильтр — одна **карта признаков (feature map)**.

PyTorch: `nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`

Пулинг (Pooling)

Агрегирует значения в окне — снижает пространственное разрешение, уменьшает число параметров, повышает устойчивость к сдвигам.

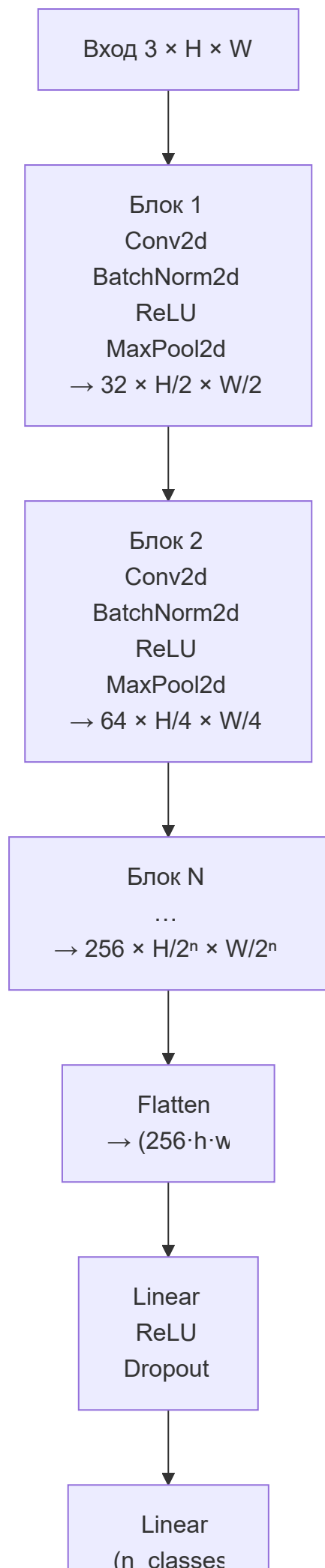


Например, при MaxPool(2×2) в каждом окне 2×2 берется максимум

$$\text{MaxPool}[i, j] = \max_{(m, n) \in \text{window}(i, j)} \text{FeatureMap}[m, n]$$

PyTorch: `nn.MaxPool2d(kernel_size=2, stride=2)` — уменьшает разрешение вдвое.

Типовая архитектура CNN



Принцип: глубже — меньше пространственное разрешение, больше каналов (feature maps). Ранние слои — края и текстуры; поздние — семантические концепции.

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2, 2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 8 * 8, 512), nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

Ключевые архитектуры

1. **LeNet-5** (1998) — первая успешная CNN; рукописные цифры
2. **AlexNet** (2012) — прорыв на ImageNet; внедрение ReLU, Dropout, использование возможностей GPU
3. **VGG-16/19** (2014) — глубина + малые ядра 3×3 как главный фактор; каскад из трех свертков 3×3 эмулирует одну 7×7 при меньшем числе параметров
4. **ResNet-50** (2015) — **residual/skip connections** — решение проблемы исчезающих градиентов (vanishing gradient) в глубоких сетях
5. **YOLO** (v1–v11) (с 2015 г. по настоящее время) — детекция объектов в реальном времени; индустриальный стандарт

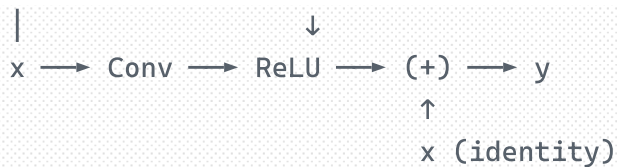
Residual Connection (ResNet)

Ключевое изобретение ResNet: «срезка» (shortcut), позволяющий градиенту течь напрямую.

Обычная сеть (без skip connection)

$x \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow y$

ResNet (с skip connection)



$$y = \mathcal{F}(x, \{W_i\}) + x$$

Благодаря этому можно обучать **сети глубиной 50–200+ слоев без деградации**.

Регуляризация нейронных сетей

Dropout

На каждой итерации обучения случайно **обнуляем** долю p нейронов (типично $p = 0.5$ для полносвязных слоев, $p = 0.1–0.2$ для Conv).

Без Dropout:

```

[o] [o] [o] [o]
  ↓  ↓  ↓  ↓
[o] [o] [o] [o]
  ↓  ↓  ↓  ↓
[o] [o] [o] [o]
  
```

С Dropout ($p=0.5$); X – обнуленные нейроны:

```

[o] [X] [o] [X]
  ↓      ↓
[o] [o] [X] [o]
          ↓
[X] [o] [o] [X]
  
```

Механизм: каждую итерацию — чуть другая подсеть, т.е. неявное ансамблирование, меньше риск «застоя» связей.

PyTorch: `nn.Dropout(p=0.5)`

⚠ Warning

`model.train()` включает Dropout, `model.eval()` — выключает.

PyTorch использует **inverted dropout**: при обучении выходы масштабируются на $\frac{1}{1-p}$, при инференсе — без изменений.

Batch Normalization

Нормализация активаций промежуточных слоев в рамках мини-батча:

$$z_i \longrightarrow \gamma_i \cdot \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta_i$$

Мини-батч (4 примера, слой из 3 нейронов):

	Нейрон 1	Нейрон 2	Нейрон 3
Пример 1:	2.1	0.3	-1.2
Пример 2:	1.8	0.9	0.5

Пример 3: 2.5 -0.1 -0.8
Пример 4: 1.6 0.7 1.1

|

↓ Нормализуем каждый нейрон по батчу: $(x - \mu) / \sigma$

↓ Затем масштабируем: $\gamma \cdot \hat{x} + \beta$ (γ, β — обучаемые)

Эффекты:

1. Предотвращает **internal covariate shift** — дрейф распределений активаций.
2. Позволяет использовать больший learning rate.

PyTorch: `nn.BatchNorm2d(num_features)` — после `Conv2d`; `nn.BatchNorm1d` — после `Linear`.

Transfer Learning (перенос обучения)

Example

Задача: классификация рентгеновских снимков (500 изображений)

Если учить ResNet-50 с нуля:

1. 25 млн параметров
2. Нужно ~1 млн изображений
3. Сотни GPU-часов
4. В результате на датасете из 500 снимков — просто катастрофическое переобучение и 0 результата

Если применить Transfer Learning:

1. Взять ResNet-50, предобученную на ImageNet (1.2M фото)
2. Ранние слои уже умеют определять края, текстуры, формы, объекты
3. Переобучить только 1 «голову» под определение 2 классов (норма/патология)
4. В результате на датасете из 500 снимков — быстрое обучение (минуты), высокое качество классификации

Transfer Learning — использование весов модели, предобученной ранее на больших наборах данных, как отправной точки.

Стратегии

1. Feature Extraction (заморозка весов)

1. Замораживаем все слои: `for param in model.parameters(): param.requires_grad = False`
2. Заменяем `model.fc` на новый слой под нужное число классов.

3. Обучаем только новый слой. Accuracy — 75-80%

2. Fine-Tuning (дообучение)

1. Размораживаем слои (часть или все).
2. Обучаем с **маленьким learning rate** ($\sim 10^{-4}$, в 10–100 раз меньше, чем с нуля)
3. Максимальное качество (Accuracy — 85-90%), требует больше данных.

🔥 Почему такой маленький learning rate

Большой lr стирает выученные признаки — происходит **катастрофическое забывание (catastrophic forgetting)**, т.к. они заменяются на случайные значения. Малый lr деликатно «тюнит» веса, не разрушая их, они просто аккуратно меняются под новую задачу.

```
import torchvision.models as models
import torch.nn as nn

model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# Feature Extraction
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 10) # обучаем только fc

# Fine-Tuning (после нескольких эпох Feature Extraction) ---
for param in model.parameters():
    param.requires_grad = True
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
```

Лабораторная работа №5

Исходные данные — датасет CIFAR-10.

60 000 цветных изображений 32×32, 10 классов: самолет, автомобиль, птица, кошка, олень, собака, лягушка, лошадь, корабль, грузовик.

Разбивка: 50 000 train / 10 000 test.

Что нужно сделать

1. Реализовать CNN в PyTorch (**Conv2d** , **MaxPool2d** , **BatchNorm2d** , **Dropout**).
2. Провести полный цикл обучения на реальном цветном датасете.
3. Применить Transfer Learning.

Что делаем

1. Загружаем CIFAR-10

```
import torchvision, torchvision.transforms as T

transform = T.Compose([T.ToTensor(), T.Normalize((0.5,0.5,0.5),
(0.5,0.5,0.5))])
trainset = torchvision.datasets.CIFAR10('./data', train=True,
download=True, transform=transform)
testset = torchvision.datasets.CIFAR10('./data', train=False,
download=True, transform=transform)
```

2. Готовим данные: `DataLoader ( batch_size=64 )`, проверяем 3-5 изображений через `matplotlib` для каждого класса (sanity check), убеждаемся, что нормализация из примера выше применена (см. строчку с `transform`)
3. Делаем сеть:

Вход: $3 \times 32 \times 32$

Блок 1:

`Conv2d(3, 32, 3, padding=1) → BatchNorm2d(32) → ReLU → MaxPool2d(2,2)`
`→  $32 \times 16 \times 16$`

Блок 2:

`Conv2d(32, 64, 3, padding=1) → BatchNorm2d(64) → ReLU → MaxPool2d(2,2)`
`→  $64 \times 8 \times 8$`

Классификатор:

`Flatten(4096) → Linear(4096,512) → ReLU → Dropout(0.5) → Linear(512,10)`

4. Обучаем: `nn.CrossEntropyLoss()`, `AdamW(lr=1e-3)`, **15+ эпох**. Сохраняем `train_loss` и `test_accuracy` после каждой эпохи. Ожидаемый результат — **65–75% Accuracy**
5. Выводим графики `train_loss` и `test_accuracy` по эпохам; Confusion matrix (`sklearn.metrics.confusion_matrix`)

| Что делаем для Tansfer Learning

1. Загружаем веса ResNet-18

```
from torchvision import models
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

 **Warning**

Добавьте `T.Resize(224)` перед `ToTensor()`, т.к. ResNet-18 обучалась на ImageNet 224×224.

Датасет все тот же CIFAR-10!

2. Делаем 5 эпох Feature Extraction

```
for p in model.parameters(): p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 10)
```

3. Делаем 10 эпох Fine-Tuning

```
for p in model.parameters(): p.requires_grad = True
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
```

4. Сверяем результаты (ожидаемая Accuracy: Feature Extraction **~75–80%**, Fine-Tuning **85–90%+**)
5. Сравниваем со своей CNN с нуля, ResNet-18 с Feature Extraction, ResNet-18 с Fine-Tuning (`test_accuracy`, сколько по времени училось)

Tip

Для воспроизводимости — `torch.manual_seed(42)`